

强化学习的数学原理与实验

Chuanwei Tang

2024-09

摘要

本文档重点在于理清强化学习背后的数学原理，追求做到在解决实际问题的过程中，如果算法效果不好，在分析原因的时候有指导思想和数学依据；必要的时候需要自己创造新的强化学习算法，此时可能需要从贝尔曼方程开始重新审查，构建随机过程并证明收敛性

在应用和构建强化学习算法的时候这些定理的条件需要满足并给出证明。例如 Soft Actor-Critic 的论文中由于在奖励中加入了信息熵，并且在强化学习过程中使用了软最大值，所以从贝尔曼方程到随机估计过程的构建都需要重新审查，并给出严格的数学证明。没有这些严格的数学证明，提出的新方法就不能保证有效且被研究社区接受。还有一个重要的原因是，当我们在工程实践中，在数学建模的过程完成后，如果这些定理都满足了，那么我们的实践便有了可靠的支撑，强化学习的算法很容易在应用中出现效果不好的情况，此时如果有了这些定理的支撑，便可以坚定的去审查代码实现或者数据是否有问题，不至于使自己淹没在巨大的不确定的潜在问题中。

本篇笔记主要参考了 [1]，在此基础上由浅入深的介绍了经典和现代强化学习算法的数学原理，并进行了代码实践¹，希望能为自己深入强化学习领域的理论和应用做好铺垫、及时总结。

¹https://github.com/LongyuanCode/rl_mathematical_cpp

目录

1 价值迭代	1
2 策略迭代	2
3 截断策略迭代	3
4 蒙特卡洛	4
4.1 蒙特卡洛树搜索	7
5 时间差分	10
6 值函数估计	12
7 策略梯度方法	17
8 结合策略梯度与价值梯度	19
8.1 Actor-Critic	19
8.2 Soft Actor-Critic	22
9 RLHF	28
9.1 LLM 的训练环节	28
9.2 RLHF 的数学原理	28
9.2.1 状态空间与动作空间的定义	29
9.2.2 Reward Model	29
9.2.3 Actor	30
9.2.4 Critic	30
9.3 PPO	31
9.4 GRPO	31
9.5 DPO	34
9.6 RLHF 的工程实践——OpenRLHF	34
10 强化学习实践指南	35

A 附录	38
A.1 收缩映射定理	38
A.2 Dvoretzky's 收敛定理	39
A.3 Robbins-Monro 定理	40
A.3.1 应用	41
A.4 拓展 Dvoretzky's 收敛定理	42
A.5 随机梯度下降 (SGD)	43
A.6 马尔可夫过程的稳态分布定理	43

1 价值迭代

用来分析最优策略和最优价值的贝尔曼最优方程

$$\begin{aligned} v(s) &= \max_{\pi(s) \in \Pi(s)} \sum_{a \in \mathcal{A}} \pi(a | s) \left(\sum_{r \in \mathcal{R}} p(r | s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v(s') \right) \\ &= \max_{\pi(s) \in \Pi(s)} \sum_{a \in \mathcal{A}} \pi(a | s) q(s, a) \end{aligned} \quad (1.1)$$

的右边是一个收缩映射 (Contraction Map)，根据收缩映射定理，智能体与环境交互的过程中， $v(s)$ 会收缩到最优的状态价值。这个过程不够直观，可以理解为函数的变化总是比自变量变化要小，那么随着自变量不断地在上一步的基础上变化，对应智能体不断的在当前状态下和环境交互，函数的变化量会逐渐变小，直至收敛到固定点 (fixed point)。

算法 1: 价值迭代

初始化: 对所有的 (s, a) ，概率模型 $p(r | s, a)$ 和 $p(s' | s, a)$ 都是已知的。

目标 : 搜索到解贝尔曼最优方程的最优状态价值和最优策略。

```

1 while 在  $\|v_k - v_{k-1}\|$  大于某个预设的小值的条件下  $v_k$  未收敛，对于第  $k$ 
   个迭代过程 do
2   for 每一个状态  $s \in \mathcal{S}$  do
3     for 每一个动作  $a \in \mathcal{A}$  do
4       q 值:  $q_k(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_k(s')$ ;
5       最大动作价值:  $a_k^*(s) = \arg \max_a q_k(s, a)$ ;
6       策略更新: 若  $a = a_k^*$  则  $\pi_{k+1}(a | s) = 1$ ，否则  $\pi_{k+1}(a | s) = 0$ ;
7       价值更新:  $v_{k+1}(s) = \max_a q_k(s, a)$ 
8     end
9   end
10 end

```

2 策略迭代

策略迭代生成了两个序列： $\{v_{\pi_0}, v_{\pi_1}, \dots, v_{\pi_k}, \dots\}$ 和 $\{\pi_0, \pi_1, \dots, \pi_k, \dots\}$ 。因为可以证明策略更新后，策略对应的价值也会被提高 [1]，所以假设 v^* 是最优的状态价值，我们有

$$v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \dots \leq v_{\pi_k} \leq \dots \leq v^*$$

算法 2: 策略迭代

初始化: 对所有的 (s, a) ，概率模型 $p(r | s, a)$ 和 $p(s' | s, a)$ 都是已知的。

初始猜测 π_0 。

目标 : 搜索最优状态价值和最优策略。

```

1 while  $v_{\pi_k}$  未收敛, 对于第  $k$  次迭代 do
    // 策略评估:
2     初始化一个任意的  $v_{\pi_k}^{(0)}$ ;
3     while  $v_{\pi_k}^{(j)}$  没有收敛, 对于第  $j$  次迭代 do
4         for 每一个状态  $s \in \mathcal{S}$  do
5              $v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a |$ 
                 $s) \left[ \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}^{(j)}(s') \right];$ 
6         end
7     end
    // 策略更新:
8     for 每一个状态  $s \in \mathcal{S}$  do
9         for 每个动作  $a \in \mathcal{A}$  do
10             $q_{\pi_k}(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s');$ 
11        end
12         $a_k^*(s) = \arg \max_a q_{\pi_k}(s, a);$ 
13        若  $a = a_k^*$ , 则  $\pi_{k+1}(a | s) = 1$ , 否则  $\pi_{k+1}(a | s) = 0$ 
14    end
15 end
```

3 截断策略迭代

从理论分析来看，价值迭代是截断策略迭代在 $j_{\text{truncate}} = 1$ 的特殊情况，而策略迭代是截断策略迭代在 $j_{\text{truncate}} = \infty$ 的特殊情况。但是在实际的编程实现中，策略迭代往往退化成截断策略迭代，因为必须设定一个终止迭代条件。

价值迭代、策略迭代、截断策略迭代都包含价值更新和策略更新这两个交替步骤，可以将这些迭代方法都看作广义策略迭代（generalization policy iteration）。

算法 3: 截断策略迭代

初始化: 对所有的 (s, a) ，概率模型 $p(r | s, a)$ 和 $p(s' | s, a)$ 都是已知的。

初始猜测 π_0 。

目标 : 搜索最优状态价值和最优策略。

```

1 while  $v_{\pi_k}$  未收敛, 对于第  $k$  次迭代 do
    // 策略评估:
2    选择一个初始猜测  $v_k^{(0)} = v_{k-1}$ , 将最大的迭代次数设置为  $j_{\text{truncate}}$  ;
3    while  $j < j_{\text{truncate}}$  do
4        for 每一个状态  $s \in \mathcal{S}$  do
5             $v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a |$ 
6                 $s) \left[ \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}^{(j)}(s') \right];$ 
7        end
8     $v_k \leftarrow v_k^{(j_{\text{truncate}})};$ 
    // 策略更新
9    for 每一个状态  $s \in \mathcal{S}$  do
10        for 每个动作  $a \in \mathcal{A}$  do
11             $q_{\pi_k}(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s');$ 
12        end
13         $a_k^*(s) = \arg \max_a q_{\pi_k}(s, a);$ 
14        若  $a = a_k^*$ , 则  $\pi_{k+1}(a | s) = 1$ , 否则  $\pi_{k+1}(a | s) = 0$ 
15    end
16 end

```

4 蒙特卡洛

Monte Carlo Exploring Starts 发生作用的数学原理是大数定律，当采样了足够多条轨迹之后，相当于每个状态动作对都会有若干条轨迹从它开始，再根据状态动作价值的定义：

$$\begin{aligned}
 q_{\pi_k}(s, a) &= \mathbb{E}[G_t \mid S_t = s, A_t = a] \\
 &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s, A_t = a] \\
 &\approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a)
 \end{aligned} \tag{4.1}$$

其中

$$g_{\pi_k}^{(i)}(s, a) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \tag{4.2}$$

基于大数定律，当 episode 的数量 n 足够大，上面对于 $q_{\pi_k}(s, a)$ 的估计就会越准确。也正是因为有大数定律的存在，使得蒙特卡洛树搜索可以被进行多线程加速——不同叶子节点的不同 Rollout 可以同时进行，都可以看作是从一个状态开始的采样轨迹；同一个叶子节点的 Rollout 和 Back Propagate 可以异步进行，从而可以以事件驱动多线程池响应机制构建蒙特卡洛树搜索。

算法 4: 蒙特卡罗基本算法（策略迭代的无模型变体）

初始化: 初始策略猜测 π_0 。

目标 : 搜索一个最优策略。

```

1 for 第  $k$  次迭代 ( $k = 0, 1, 2, \dots$ ) do
2   for 每一个状态  $s \in \mathcal{S}$  do
3     for 每一个动作  $a \in \mathcal{A}(s)$  do
4       收集从  $(s, a)$  出发、遵循  $\pi_k$  执行的足够多的回合;
5       // 策略评估:
6        $q_{\pi_k}(s, a) \approx q_k(s, a) =$  从  $(s, a)$  出发的所有回合的平均回报;
7     end
8     // 策略改进:
9      $a_k^*(s) = \arg \max_a q_k(s, a);$ 
10     $\pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & \text{otherwise} \end{cases};$ 
11  end
12 end

```

算法 5: MC 探索性出发 (MC Basic 的高效变体)

初始化: 对所有 (s, a) , 初始化策略 $\pi_0(a | s)$ 和动作价值 $q(s, a)$ 。设置 $\text{Returns}(s, a) = 0$ 且 $\text{Num}(s, a) = 0$ 。**目标** : 搜索最优策略。

```

1 for 每个回合 do
2   回合生成: 选择起始状态-动作对  $(s_0, a_0)$  并确保所有可能的状态-动作
      对都能被选中 (探索性出发条件)。根据当前策略生成长度为  $T$  的回合:
       $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 。;
3   回合初始化:  $g \leftarrow 0$ ;
4   for 回合中的每一步  $t = T - 1, T - 2, \dots, 0$  do
5      $g \leftarrow \gamma g + r_{t+1}$ ;
6      $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$ ;
7      $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$ ;
      // 策略评估:
8      $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$ ;
      // 策略改进:
9      $\pi(a | s_t) = 1$  (若  $a = \arg \max_a q(s_t, a)$ ), 否则  $\pi(a | s_t) = 0$ ;
10  end
11 end

```

算法 6: MC ϵ -贪心 (MC 探索性出发的变体)

初始化: 对所有 (s, a) , 初始化策略 $\pi_0(a | s)$ 和动作价值 $q(s, a)$ 。

设置 $\text{Returns}(s, a) = 0$ 且 $\text{Num}(s, a) = 0$ 。

设定 $\epsilon \in (0, 1]$ 。

目标 : 搜索最优策略。

```

1 for 每个回合 do
2   回合生成: 选择起始状态-动作对  $(s_0, a_0)$  (无需探索性出发条件)。根
   据当前策略生成长度为  $T$  的回合:
3    $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 。 ;
   // 回合初始化
4    $g \leftarrow 0$ ;
5   for 回合中的每一步  $t = T - 1, T - 2, \dots, 0$  do
6      $g \leftarrow \gamma g + r_{t+1}$ ;
7      $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$ ;
8      $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$ ;
     // 策略评估:
9      $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$ ;
     // 策略改进:
10    令  $a^* = \arg \max_a q(s_t, a)$ , 并更新策略:
11     $\pi(a | s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|}\epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|}\epsilon, & a \neq a^* \end{cases}$ ;
12  end
13 end

```

4.1 蒙特卡洛树搜索

UTC 算法中选择最优子节点用的是 [2]

$$\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}} \quad (4.3)$$

如果用并行, 可以用四个线程全部探索四个子节点, 返回的仿真回报可以直接

加在一起求均值。其背后的数学原理也是上述大数定律，假设从每个节点出发有 n 条轨迹，第 i 条轨迹的回报是

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a] \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a) \quad (4.4)$$

算法 7: UCT 算法

```

1 Function UCTSearch( $s_0$ )
2   创建根节点  $v_0$ ，状态为  $s_0$ ;
3   while 在计算预算范围内 do
4      $v_l \leftarrow \text{TreePolicy}(v_0)$ ;
5      $\Delta \leftarrow \text{DefaultPolicy}(s(v_l))$ ;
6     Backup( $v_l, \Delta$ );
7   end
8   return  $a(\text{BestChild}(v_0, 0))$ 

9 Function TreePolicy( $v$ )
10  while  $v$  是非终止节点 do
11    if  $v$  未被完全扩展 then
12      return Expand( $v$ )
13    end
14    else
15       $v \leftarrow \text{BestChild}(v, C_p)$ ;
16    end
17  end
18  return  $v$ 

19 Function Expand( $v$ )
20  从  $A(s(v))$  中选择一个未尝试的动作  $a$ ;
21  添加一个新子节点  $v'$  到  $v$ ;
22  设置  $s(v') = f(s(v), a)$ ;
23  设置  $a(v') = a$ ;
24  return  $v'$ 

25 Function BestChild( $v, c$ )
26  return  $\arg \max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c\sqrt{\frac{2 \ln N(v)}{N(v)}}$ 

27 Function DefaultPolicy( $s$ )
28  while  $s$  是非终止状态 do
29    从  $A(s)$  中均匀随机选择一个动作  $a$ ;
30     $s \leftarrow f(s, a)$ ;
31  end
32  return 状态  $s$  的奖励

33 Function Backup( $v, \Delta$ )
34  while  $v$  不为空 do
35     $N(v) \leftarrow N(v) + 1$ ;
36     $Q(v) \leftarrow Q(v) + \Delta(v, p)$ ;
37     $v \leftarrow v$  的父节点;
38  end

```

算法 8: UCT 双人博弈回溯 (Negamax 版本)

```
1 Function BACKUPNEGAMAX( $v$ ,  $\Delta$ )
2   while  $v$  不为空 do
3      $N(v) \leftarrow N(v) + 1$  ;           // 更新节点访问次数
4      $Q(v) \leftarrow Q(v) + \Delta$  ;      // 累积动作价值
5      $\Delta \leftarrow -\Delta$  ;          // Negamax 反转增量
6      $v \leftarrow v$  的父节点 ;          // 回溯到父节点
7   end
```

5 时间差分

sarsa 算法的有效性来自于 Dvoretzky' s 收敛定理。可以证明其算法的运行过程满足 Dvoretzky' s 收敛定理中的随机过程。

算法 9: Sarsa 算法最优策略学习

初始化: 对所有 (s, a) 和所有 t , 设置 $\alpha_t(s, a) = \alpha > 0$ 。

设置 $\epsilon \in (0, 1)$ 。

初始化所有 (s, a) 的 $q_0(s, a)$ 。

根据 q_0 生成初始 ϵ -贪心策略 π_0 。

目标 : 学习能够使智能体从初始状态 s_0 到达目标状态的最优策略。

```

1 for 每个回合 do
2   根据  $\pi_0(s_0)$  在  $s_0$  生成动作  $a_0$ ;
3   while  $s_t (t = 0, 1, 2, \dots)$  不是目标状态 do
4     收集经验样本  $(r_{t+1}, s_{t+1}, a_{t+1})$ :
5     通过与环境交互生成  $r_{t+1}, s_{t+1}$ ; 根据  $\pi_t(s_{t+1})$  生成  $a_{t+1}$ ;
6     更新  $(s_t, a_t)$  的  $q$  值:  $q_{t+1}(s_t, a_t) =$ 
         $q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))]$ 
7     更新  $s_t$  的策略:
        
$$\pi_{t+1}(a|s_t) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1), & a = \arg \max_a q_{t+1}(s_t, a) \\ \frac{\epsilon}{|\mathcal{A}(s_t)|}, & \text{其他情况} \end{cases};$$

8      $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$ ;
9   end
10 end
```

Q-learning 算法本质上是对下面等式的随机估计算法:

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a_{t+1}} q(S_{t+1}, a_{t+1}) \mid S_t = s, A_t = a \right] \quad (5.1)$$

这是贝尔曼最优方程的动作价值表达形式。Q-learning 算法的随机过程为：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - \left(r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} q_t(s_{t+1}, a) \right) \right]$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \text{for all } (s, a) \neq (s_t, a_t)$$
(5.2)

其收敛性证明可借助于 Robbins Monro 定理。

算法 10: Q-learning 算法 (on-policy 版本) 最优策略学习

初始化： 对所有 (s, a) 和 t , 设置 $\alpha_t(s, a) = \alpha > 0$ 。设置 $\epsilon \in (0, 1)$ 。初始化所有 (s, a) 的 $q_0(s, a)$ 。根据 q_0 生成初始 ϵ -贪心策略 π_0 。

目标： 学习能够使智能体从初始状态 s_0 到达目标状态的最优路径。

```

1 for 每个回合 do
2   while  $s_t(t = 0, 1, 2, \dots)$  不是目标状态 do
3     收集经验样本  $(a_t, r_{t+1}, s_{t+1})$ : 根据  $\pi_t(s_t)$  生成  $a_t$ , 通过与环境交互生成
         $r_{t+1}, s_{t+1}$ ;
4     更新  $(s_t, a_t)$  的  $q$  值:  $q_{t+1}(s_t, a_t) =$ 
         $q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))];$ 
5     更新  $s_t$  的策略:
        
$$\pi_{t+1}(a|s_t) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1), & a = \arg \max_a q_{t+1}(s_t, a); \\ \frac{\epsilon}{|\mathcal{A}(s_t)|}, & \text{其他情况} \end{cases};$$

6      $s_t \leftarrow s_{t+1}$ ;
7   end
8 end
```

算法 11: Q-learning 算法 (off-policy 版本) 最优策略学习

初始化： 对所有 (s, a) 初始化 $q_0(s, a)$; 定义行为策略 $\pi_b(a|s)$; 对所有的 (s, a) 和 t 设置 $\alpha_t(s, a) = \alpha > 0$ 。

目标： 从 π_b 生成的样本中学习最优目标策略 π_T 。

```

1 for 由  $\pi_b$  生成的每个回合  $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$  do
2   for 回合中的每一步  $t = 0, 1, 2, \dots$  do
3     更新  $(s_t, a_t)$  的  $q$  值:  $q_{t+1}(s_t, a_t) =$ 
         $q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))];$ 
4     更新目标策略:  $\pi_{T,t+1}(a|s_t) = \begin{cases} 1, & a = \arg \max_a q_{t+1}(s_t, a); \\ 0, & \text{其他情况} \end{cases};$ 
5   end
6 end
```

6 值函数估计

强化学习框架下智能体对环境的状态价值估计问题的函数估计方式可以建模为找到最优的参数 w 使得 $\hat{v}(s, w)$ 是对每个状态 s 的价值 $v_\pi(s)$ 最佳估计，目标函数：

$$J(w) = \mathbb{E} [(v_\pi(S) - \hat{v}(S, w))^2] \quad (6.1)$$

如果每个状态都会被同样的重视，那么公式(6.1)中的期望就可以用均匀分布的概率乘积和计算。但是智能体一般都有一定的策略去和环境进行交互，并且假设该过程是一个马尔可夫过程，在这个前提下，可以引入稳态分布来描述马尔可夫决策过程的长期行为。具体而言，在智能体在一个给定策略指导下执行了足够长的时间以后，智能体在任何一个状态下的概率可以被这个稳态分布 d_π 描述，于是(6.1)可以近似为

$$J(w) = \sum_{s \in \mathcal{S}} d_\pi(s) (v_\pi(s) - \hat{v}(s, w))^2 \quad (6.2)$$

稳态分布分布 d_π 不需要知道其确切的值，但是正是因为它的存在，可以将(6.1)中的随机变量 S 写成一个采样值 s_t ，从而引入了随机梯度下降法。为了最小化(6.1)中的目标函数，可以构建随机梯度下降法的随机过程（随机梯度下降法是一个特殊的 Robbins Monro 算法）

$$w_{k+1} = w_k - \alpha \nabla_w J(w_k) \quad (6.3)$$

其中

$$\begin{aligned} \nabla_w J(w_k) &= \nabla_w \mathbb{E} [(v_\pi(S) - \hat{v}(S, w_k))^2] \\ &= \mathbb{E} [\nabla_w (v_\pi(S) - \hat{v}(S, w_k))^2] \\ &= 2\mathbb{E} [(v_\pi(S) - \hat{v}(S, w_k)) (-\nabla_w \hat{v}(S, w_k))] \\ &= -2\mathbb{E} [(v_\pi(S) - \hat{v}(S, w_k)) \nabla_w \hat{v}(S, w_k)] \end{aligned} \quad (6.4)$$

于是随机梯度下降法可以写成

$$w_{k+1} = w_k + 2\alpha_k \mathbb{E} [(v_\pi(S) - \hat{v}(S, w_k)) \nabla_w \hat{v}(S, w_k)] \quad (6.5)$$

基于上面的稳态分布的论述，进一步可以写成

$$w_{k+1} = w_k + \alpha_k ((v_\pi(s_t) - \hat{v}(s_t, w_k)) \nabla_w \hat{v}(s_t, w_k)) \quad (6.6)$$

不过此时我们还不知道 $v_\pi(s_t)$ ，但是可以借助蒙特卡洛和时间差分的方法进行估计：

1. 蒙特卡洛

$$w_{t+1} = w_t + \alpha_t (g_t - \hat{v}(s_t, w_t)) \nabla_w \hat{v}(s_t, w_t) \quad (6.7)$$

2. 时间差分

$$w_{t+1} = w_t + \alpha_t (r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)) \nabla_w \hat{v}(s_t, w_t) \quad (6.8)$$

算法 12: 基于函数逼近的 TD 状态值学习

初始化: 对 w 可微的函数 $\hat{v}(s, w)$ ；初始化参数 w_0 。

目标 : 学习给定策略 π 的真实状态值。

```

1 for 由  $\pi$  生成的每个回合  $\{(s_t, r_{t+1}, s_{t+1})\}_t$  do
2   for 每个样本  $(s_t, r_{t+1}, s_{t+1})$  do
3     一般情况:
4        $w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t);$ 
5     线性情况:  $w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \phi^T(s_{t+1})w_t - \phi^T(s_t)w_t] \phi(s_t);$ 
6   end
7 end

```

算法 13: 基于函数逼近的 Sarsa 算法

初始化: 初始化参数 w_0 ; 初始策略 π_0 ; 对所有 t 设置 $\alpha_t = \alpha > 0$; 设置 $\epsilon \in (0, 1)$ 。

目标 : 学习能使智能体从初始状态 s_0 到达目标状态的最优策略。

```

1 for 每个回合 do
2   根据  $\pi_0(s_0)$  在  $s_0$  生成动作  $a_0$ ;
3   while  $s_t$  ( $t = 0, 1, 2, \dots$ ) 不是目标状态 do
4     收集经验样本  $(r_{t+1}, s_{t+1}, a_{t+1})$ : 通过与环境交互生成  $r_{t+1}, s_{t+1}$ ;
5     根据  $\pi_t(s_{t+1})$  生成  $a_{t+1}$ ;
6     更新  $q$  值:  $w_{t+1} =$ 
         $w_t + \alpha_t [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t);$ 
7     更新策略:  $\pi_{t+1}(a|s_t) =$ 
        
$$\begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1), & a = \arg \max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1}); \\ \frac{\epsilon}{|\mathcal{A}(s_t)|}, & \text{其他情况} \end{cases};$$

8      $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1};$ 
9   end
10 end

```

参考 Q-learning 的数学建模过程, 深度 Q-learning 的目标是最小化

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, a, w) \right)^2 \right] \quad (6.9)$$

参考前述贝尔曼最优方程的动作价值形式, 这个目标函数可以看作平方贝尔曼最优误差。又由于

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a) \mid S_t = s, A_t = a \right], \quad \text{for all } s, a \quad (6.10)$$

所以 J 的期望值应该等于 0, 于是可以通过 SGD 最小化 J 。考虑到实际应用的时候, 动作空间大小 $|\mathcal{A}(S')|$ 可能会很大, 神经网络可能会比较复杂, 所以 $\max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a)$ 计算可能代价比较大, 为了方便计算, 可以认为在比较小的事件片段 C 内, 这个值是不变的。另外, $\max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a)$ 通常不是一个凸

运算，所以神经网络训练会不稳定。

需要仔细理解这里的回放缓冲区 $\mathcal{B}$ 的设计、网络参数异步更新的设计，这些新的设计不能破坏 SGD 的收敛性。回顾定理 6.4，随机梯度下降法的收敛性定理：

对于随机梯度下降算法

$$w_{k+1} = w_k - \alpha \nabla_w f(w_k, x_k)$$

如果下列条件满足

- (a) $0 < c_1 \leq \nabla_w^2 f(w, X) \leq c_2$;
- (b) $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$;
- (c) $\{x_k\}_{k=1}^{\infty}$ are i.i.d.

那么 w_k 将会几乎必然 (almost surely) 收敛到方程 $\nabla_w \mathbb{E}[f(w, X)] = 0$ 的根。

缓冲区中选取数据的时候使用了随机采样，在 MDP 中交互采样得到的数据本身不满足独立假设，因为这一时刻的状态和上一时刻的状态有关。非独立同分布的数据对训练神经网络有很大的影响，会使神经网络拟合到最近训练的数据上。缓冲区回放可以打破样本之间的相关性，让其满足独立假设；两套神经网络参数的异步更新本身不会影响条件 (b)、(c)，对于条件 (a)，我们假设

$$y(w_t) \doteq r_{t+1} + \gamma \max_{a_{t+1} \in \mathcal{A}(s')} \hat{q}(s', a, w_t)$$

$$y(w_T) \doteq r_{t+1} + \gamma \max_{a_{t+1} \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

$$\delta = y(w_t) - y(w_T)$$

于是引入两套神经网络的目标函数可以写成

$$J = \left(r_{t+1} + \gamma \max_{a_{t+1} \in \mathcal{A}(s')} \hat{q}(s', a, w_t) - \delta - \hat{q}(s_t, a_t, w_t) \right)^2 \quad (6.11)$$

不难看出条件 (a) 仍然满足。

算法 14: 深度 Q 学习算法 (off-policy 版本)

初始化: 初始化主网络和目标网络, 参数相同。

目标 : 通过学习逼近最优动作值函数的目标网络, 从行为策略 π_b 生成的样本中学习。

```

1 将  $\pi_b$  生成的样本存储到经验回放缓存  $\mathcal{B} = \{(s, a, r, s')\}$ ;
2 for 每次迭代 do
3   从  $\mathcal{B}$  中均匀采样一个小批量样本;
4   for 每个样本  $(s, a, r, s')$  do
5     计算目标值:  $y_T = r + \gamma \max_{a \in A(s')} \hat{q}(s', a, w_T)$ , 其中  $w_T$  是目标网
       络参数;
6     更新主网络以最小化  $(y_T - \hat{q}(s, a, w))^2$ ;
7   end
8   每  $C$  次迭代设置  $w_T = w$ ;
9 end

```

7 策略梯度方法

“一个好的策略应该能够使得状态的价值最大化”，这句话是策略梯度方法的核心思想。优化的目标有如下两类及其不同但是等价的表达形式：

<i>Metric</i>	<i>Expression1</i>	<i>Expression2</i>	<i>Expression3</i>
$\bar{v}_\pi$	$\sum_{s \in \mathcal{S}} d(s) v_\pi(s)$	$\mathbb{E}_{S \sim d} [v_\pi(S)]$	$\lim_{n \rightarrow \infty} \mathbb{E} [\sum_{t=0}^n \gamma^t R_{t+1}]$
$\bar{r}_\pi$	$\sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s)$	$\mathbb{E}_{S \sim d_\pi} [r_\pi(S)]$	$\lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E} [\sum_{t=0}^{n-1} R_{t+1}]$

策略梯度方法的优化目标的梯度有如下统一形式：

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a | s, \theta) q_\pi(s, a) \\ &= \mathbb{E}_{s \sim \eta, A \sim \pi(S, \theta)} [\nabla_\theta \ln \pi(A | S, \theta) q_\pi(S, A)] \end{aligned} \quad (7.1)$$

其中 η 是状态的分布。可以证明，当使用 $\bar{r}_\pi$ 或者 $\bar{v}_\pi$ 作为优化目标时， η 就是 $d_\pi(s)$ ：

$$\begin{aligned} \nabla_\theta \bar{r}_\pi &= (1 - \gamma) \nabla_\theta \bar{v}_\pi \approx \sum_{s \in \mathcal{S}} d_\pi(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a | s, \theta) q_\pi(s, a) \\ &= \mathbb{E} [\nabla_\theta \ln \pi(A | S, \theta) q_\pi(S, A)] \end{aligned} \quad (7.2)$$

其中 $S \sim d_\pi$ 且 $A \sim \pi(S, \theta)$ 。优化 $J(\theta) = \bar{r}_\pi$ 的随机梯度方法为

$$\begin{aligned} \theta_{t+1} &= \theta_t + \alpha \nabla_\theta J(\theta_t) \\ &= \theta_t + \alpha \mathbb{E} [\nabla_\theta \ln \pi(A | S, \theta_t) q_\pi(S, A)] \\ &\approx \theta_t + \alpha \nabla_\theta \ln \pi(a_t | s_t, \theta_t) q_t(s_t, a_t) \end{aligned}$$

$q_t(s_t, a_t)$ 的计算借助于蒙特卡洛方法，根据状态动作价值的定义：

$$\begin{aligned} q_{\pi_k}(s, a) &= \mathbb{E} [G_t | S_t = s, A_t = a] \\ &= \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s, A_t = a] \\ &\approx \underbrace{r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots}_{q_k(s, a)} \end{aligned}$$

在数学上其收敛性由大数定律保证。

算法 15: 蒙特卡洛策略梯度算法 (REINFORCE)

初始化: 初始化参数 θ ; 折扣因子 $\gamma \in (0, 1)$; 学习率 $\alpha > 0$ 。

目标 : 学习最优策略以最大化 $J(\theta)$ 。

```

1 for 每个回合 do
2   根据策略  $\pi(\theta)$  生成回合轨迹  $\{s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T\}$ ;
3   for  $t = 0, 1, \dots, T - 1$  do
4     价值更新:  $q_t(s_t, a_t) = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$ ;
5     策略更新:  $\theta \leftarrow \theta + \alpha \nabla_{\theta} \ln \pi(a_t | s_t, \theta) q_t(s_t, a_t)$ ;
6   end
7 end

```

8 结合策略梯度与价值梯度

Actor-Critic 算法结合了策略梯度和价值梯度的方法。这一类方法都有统一的优化目标 $J(\theta)$ 和随机梯度方法的优化过程：

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_{\theta} J(\theta_t) \\ &= \theta_t + \alpha \mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \ln \pi(A | S, \theta_t) q_{\pi}(S, A)] \\ &\Downarrow \text{随机梯度估计} \\ \theta_{t+1} &= \theta_t + \alpha \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) q_t(s_t, a_t)\end{aligned}$$

不同算法的核心区别在于不同的场景下优化目标 $J(\theta)$ 的具体形式的不同以及随机梯度估计的方法不同。

8.1 Actor-Critic

QAC 算法中的动作价值函数的更新采用了前述的值函数估计方法，策略更新使用了前述的策略梯度方法。

算法 16: 基础 Actor-Critic 家算法 (QAC)

初始化: 策略函数 $\pi(a|s, \theta_0)$ ，初始参数 θ_0 ；价值函数 $q(s, a, w_0)$ ，初始参数 w_0 ；学习率 $\alpha_w, \alpha_{\theta} > 0$ 。

目标 : 学习最优策略以最大化 $J(\theta)$ 。

- 1 在每个回合的时间步 t 进行以下步骤：
- 2 根据 $\pi(a|s_t, \theta_t)$ 生成 a_t ，观测 r_{t+1}, s_{t+1} ；
- 3 根据 $\pi(a|s_{t+1}, \theta_t)$ 生成 a_{t+1} ；
- 4 *Actor* (策略更新): $\theta_{t+1} = \theta_t + \alpha_{\theta} \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) q(s_t, a_t, w_t)$ ；
- 5 *Critic* (价值更新):

$$w_{t+1} = w_t + \alpha_w [r_{t+1} + \gamma q(s_{t+1}, a_{t+1}, w_t) - q(s_t, a_t, w_t)] \nabla_w q(s_t, a_t, w_t);$$

为了减少随机梯度估计的方差，增强算法的稳定性，A2C 算法引入了一个 baseline $b(s)$ ，可以证明，最优的 $b(s)$ 为

$$b^*(s) = \frac{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \ln \pi(A | s, \theta_t)\|^2 q_{\pi}(s, A)]}{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \ln \pi(A | s, \theta_t)\|^2]}, \quad s \in S \quad (8.1)$$

为了计算方便将权重 $\|\nabla_{\theta} \ln \pi(A | s, \theta_t)\|^2$ 移除, 得到次优的 $b(s)$

$$b^{\dagger}(s) = \mathbb{E}_{A \sim \pi} [q_{\pi}(s, A)] = v_{\pi}(s) \quad (8.2)$$

此时随机梯度算法为

$$\begin{aligned} \theta_{t+1} &= \theta_t + \alpha \mathbb{E} [\nabla_{\theta} \ln \pi(A | S, \theta_t) [q_{\pi}(S, A) - v_{\pi}(S)]] \\ &\doteq \theta_t + \alpha \mathbb{E} [\nabla_{\theta} \ln \pi(A | S, \theta_t) \delta_{\pi}(S, A)] \end{aligned}$$

其中

$$\delta_{\pi}(S, A) \doteq q_{\pi}(S, A) - v_{\pi}(S)$$

算法 17: Advantage Actor-Critic 算法 (A2C/TD Actor-Critic)

初始化: 策略函数 $\pi(a|s, \theta_0)$ (初始参数 θ_0); 价值函数 $v(s, w_0)$ (初始参数 w_0); 学习率 $\alpha_w, \alpha_{\theta} > 0$ 。

目标 : 学习最优策略以最大化 $J(\theta)$ 。

- 1 在每个回合的时间步 t 进行以下步骤:
 - 2 根据 $\pi(a|s_t, \theta_t)$ 生成 a_t , 观测 r_{t+1}, s_{t+1} ;
 - 3 优势函数 (TD 误差): $\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$;
 - 4 Actor (策略更新): $\theta_{t+1} = \theta_t + \alpha_{\theta} \delta_t \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t)$;
 - 5 Critic (价值更新): $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$;
-

前面的算法都是 on-policy 的方式, 当**我们希望用一个策略指导下的智能体与环境交互的样本去优化另一个策略**, 就需要借助于 off-policy 的方式, 其优化目标变成

$$J(\theta) = \sum_{s \in \mathcal{S}} d_{\beta}(s) v_{\pi}(s) = \mathbb{E}_{S \sim d_{\beta}} [v_{\pi}(S)]$$

其中 $d_{\beta}(s)$ 是策略 β 下的稳态分布而 v_{π} 是策略 π 下的状态价值。**借助于 importance sampling 可以用行为策略 β 下的状态的稳态分布估计目标策略 π 的状态的稳态分布**, 其梯度为

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{S \sim \rho, A \sim \beta} \left[\underbrace{\frac{\pi(A|S, \theta)}{\beta(A|S)}}_{\text{importance weight}} \nabla_{\theta} \ln \pi(A|S, \theta) q_{\pi}(S, A) \right] \quad (8.3)$$

其中状态分布 ρ 为

$$\rho(s) \doteq \sum_{s' \in \mathcal{S}} d_\beta(s') \Pr(s | s'), \quad s \in \mathcal{S}$$

其中 $\Pr(s | s') = \sum_{k=0}^{\infty} \gamma^k [P_\pi^k]_{s's} = [(I - \gamma P_\pi)^{-1}]_{s's}$ 是在策略 π 下从状态 s' 转移向 s 的衰减全概率。于是，再结合前述的减小随机估计方差的次优 baseline 的方法我们有

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\frac{\pi(A|S, \theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S, \theta) q_\pi(S, A) - v_\pi(S) \right]$$

相应的随机梯度算法为

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t, \theta_t)}{\beta(a_t|s_t)} \nabla_\theta \ln \pi(a_t|s_t, \theta_t) (q_t(s_t, a_t) - v_t(s_t))$$

类似于前述方法，这其中的 $q_t(s_t, a_t) - v_t(s_t)$ 可以用时间差分误差代替，即

$$q_t(s_t, a_t) - v_t(s_t) \approx r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t) \doteq \delta_t(s_t, a_t)$$

算法 18: 基于重要性采样的 off-policy Actor-Critic 算法

初始化: 给定行为策略 $\beta(a|s)$; 目标策略 $\pi(a|s, \theta_0)$ (初始参数 θ_0); 价值函数 $v(s, w_0)$ (初始参数 w_0); 学习率 $\alpha_w, \alpha_\theta > 0$ 。

目标 : 学习最优策略以最大化 $J(\theta)$ 。

- 1 在每个回合的时间步 t 进行以下步骤:
 - 2 根据 $\beta(s_t)$ 生成 a_t , 观测 r_{t+1}, s_{t+1} ;
 - 3 优势函数 (TD 误差): $\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$;
 - 4 Actor (策略更新): $\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t, \theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t)$;
 - 5 Critic (价值更新): $w_{t+1} = w_t + \alpha_w \frac{\pi(a_t|s_t, \theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_w v(s_t, w_t)$;
-

前述所有算法中 on-policy 的方式样本效率会比较低; 另外, 如果是概率策略, 那么能处理的动作空间是有限的, 因为概率策略需要从所有的动作概率分布上采样, 虽然可以将无限动作空间离散化, 但是这样做难免会粗糙。于是, off-policy 的、确定性策略 (Deterministic Policy) 的算法被提出来。所谓的确定性策略是指某个状态下, 只有一个动作以 1 的概率被选择, 其他的动作被选择的概率为零。

优化目标为确定性策略 μ 的 Average Reward $\bar{r}_\mu$:

$$\begin{aligned} J(\theta) &= \bar{r}_\mu = \sum_{s \in \mathcal{S}} d_\mu(s) r_\mu(s) \\ &= \mathbb{E}_{S \sim d_\mu} [r_\mu(S)] \end{aligned}$$

从优化目标中可以看出稳态分布只和 μ 有关，不借助于行为策略的采样去估计目标分布，所以下面的算法中没有使用 importance sampling。其随机梯度算法为

$$\begin{aligned} \theta_{t+1} &= \theta_t + \alpha_\theta \mathbb{E}_{S \sim d_\mu} \left[\nabla_\theta \mu(S) (\nabla_a q_\mu(S, a)) \Big|_{a=\mu(S)} \right] \\ &\approx \theta_t + \alpha_\theta \nabla_\theta \mu(s_t) (\nabla_a q_\mu(s_t, a)) \Big|_{a=\mu(s_t)} \end{aligned}$$

其中 μ 是确定性策略。

算法 19: 确定性策略梯度算法 (确定性 Actor-Critic)

初始化: 给定行为策略 $\beta(a|s)$; 确定性目标策略 $\mu(s, \theta_0)$ (初始参数 θ_0);

价值函数 $q(s, a, w_0)$ (初始参数 w_0); 学习率 $\alpha_w, \alpha_\theta > 0$ 。

目标 : 学习最优策略以最大化 $J(\theta)$ 。

- 1 在每个回合的时间步 t 执行:
 - 2 根据 β 生成 a_t , 观测 r_{t+1}, s_{t+1} ;
 - 3 TD 误差: $\delta_t = r_{t+1} + \gamma q(s_{t+1}, \mu(s_{t+1}, \theta_t), w_t) - q(s_t, a_t, w_t)$;
 - 4 Actor (策略更新): $\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu(s_t, \theta_t) (\nabla_a q(s_t, a, w_t)) \Big|_{a=\mu(s_t)}$;
 - 5 Critic (价值更新): $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w q(s_t, a_t, w_t)$;
-

8.2 Soft Actor-Critic

在具体的应用中，动作空间和状态空间往往会有多模态的形式，例如自动驾驶中动作有加速度、方向盘转角两个模态，在不同的驾驶场景中状态空间也可能有多种模态。传统的强化学习只能收敛到选择最大的动作价值的策略，虽然可以用类似 ε 贪婪的方法探索一些非最大价值的动作，但是这个方法相比于在价值中引入了熵的强化学习方法来说有诸多弊端。其中最重要的，便是对于**多模态的 Q 值**情况下，soft Q-learning 可以学习到直接对应于 $Q(S, A)$ 分布的策略函数。最大熵强化学习方法在奖励中加入了策略函数的熵，熵在信息论的领域中是对不确定性的

度量，熵的引入增强了策略的探索性。在奖励中引入熵的强化学习优化过程如下

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} \left[\sum_t R(s_t, a_t) + \alpha H(\pi(\cdot | s_t)) \right] \quad (8.4)$$

其中 $H(\cdot) = -\ln(\cdot)$ 是信息熵。可以单独考察 $\sum_t \alpha H(\pi(\cdot | s_t))$ 获得 intuition: 在一个 episode 中, 要想使这个值比较大, 同时又要满足 $\sum_{a \in \mathcal{A}} \pi(a | s_t) = 1$, 那么每个时刻对每个动作的选择概率应当尽量都接近 0 但是总和为 1, 故而熵的引入增强了策略的探索性。注意! 对于连续的动作空间, 这里的 π 实际上是动作概率密度, 这一点在 SAC 原论文 [3]、OpenAI Spinning Up[4] 都有明确支持, 网上很多公开资料关于这一块表述或者理解是有问题的。对于连续型随机变量动作 A , 其在某个点 a_t 处的概率是 0, 所以这里的 $\pi(a_t | s_t)$ 也自然应该是概率密度, 不然它的值就一直是零了。搞清楚这个概念, 在后面对 a_t 进行非线性压缩变换到 $(-1, 1)$ 的时候, 其概率密度 π 也需要借助概率密度变换定理进行修正, 如果这个概念没有理清, 会很奇怪 “为什么会将概率密度当作概率”。

引入了信息熵的动作价值为

$$Q_{soft}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1}, a_{t+1} \sim \mu} [Q_{soft}(s_{t+1}, a_{t+1}) - \alpha \ln \pi(a_{t+1} | s_{t+1})] \quad (8.5)$$

又参考

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim \rho} [V(s_{t+1})]$$

所以可以得到

$$V_{soft}(s_t) = \mathbb{E}_{a_t \sim \pi} [Q_{soft}(s_t, a_t) - \alpha \ln \pi(a_t | s_t)] \quad (8.6)$$

在传统的 Q-learning 过程中, 根据贝尔曼最优方程, 我们有

$$V(s_t) = \max_a Q(s_t, a)$$

在多模态的 Q 值中如果使用传统的 Q-learning 方法, 那么获得的策略函数可能只对一种模态有较好的效果。如图1中1a所示:

于是在 SAC 使用的 Soft Q-learning 中, 我们使用 $\max(\cdot)$ 的平滑版本 $\text{soft max}(\cdot)$

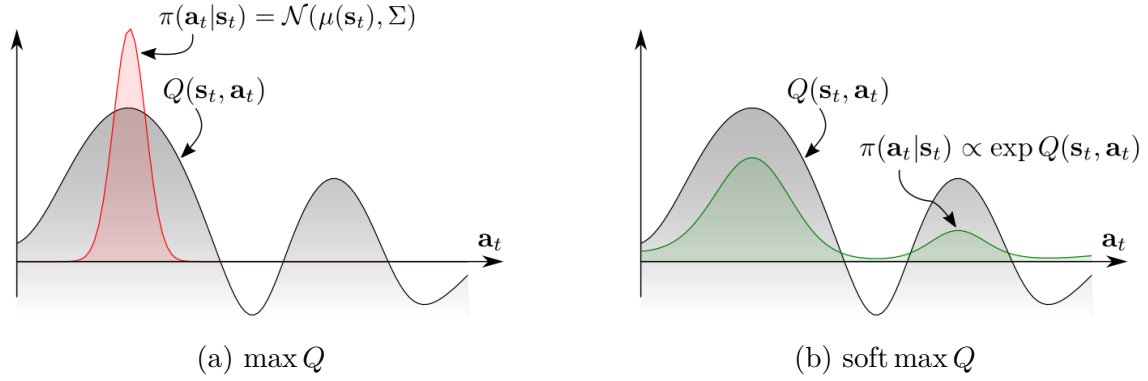


图 1: Actor-Critic 和 Soft Actor-Critic 本质区别 [5]

——这个操作是凸的但不是严格凸的

$$V_{soft}(s_t) = \text{soft max}(Q(s_t, a)) = -\frac{1}{\alpha} \ln \sum_a \exp \left(-\frac{1}{\alpha} Q_{soft}(s_t, a) \right) \quad (8.7)$$

直接获得对应于 $Q(S, A)$ 分布的策略函数，如上图右边。为了能够达到策略分布直接对应于 $Q(S, A)$ 的分布特征，可以借助玻尔兹曼分布，并令其能量函数为 $-1/\alpha \cdot Q_{soft}(s_t, a_t)$ ，进而

$$\pi(a_t|s_t) \propto \exp \left(-\frac{1}{\alpha} Q_{soft}(s_t, a_t) \right) \quad (8.8)$$

然后，综合考虑动作价值与状态价值的关系、上述 $V_{soft}(s_t)$ 的计算过程、所有动作概率和为 1 不难写出

$$\pi(s_t, a_t) = \frac{\exp \left(-\frac{1}{\alpha} Q_{soft}(s_t, a_t) \right)}{\sum_a \exp \left(-\frac{1}{\alpha} Q_{soft}(s_t, a) \right)}$$

有了上面的分析，不难写出 Soft 贝尔曼方程：

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in S} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' | s') (q_\pi(s', a') - \alpha \ln(\pi(a' | s'))) \quad (8.9)$$

基于贝尔曼方程，便可以使用策略评估和策略改善的框架迭代求解最优的策略和最优的动作价值。

根据定义我们有

$$q_\pi(s, a) = \sum_{r \in \mathcal{R}} p(r | s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) v_\pi(s')$$

我们可以写出 Q_{soft} 场景下的优化目标:

$$J_Q(\psi) = \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[\frac{1}{2} (Q_\psi(s_t, a_t) - (r(s_t, a_t) + \gamma V_\theta(s_{t+1})))^2 \right] \quad (8.10)$$

又根据上述等式(8.7), 优化目标又可以写成

$$J_Q(\psi) = \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}, a_{t+1} \sim \pi_\phi} \left[\frac{1}{2} \left(Q_\psi(s_t, a_t) - \left(r(s_t, a_t) + \gamma Q_\theta(s_{t+1}, a_{t+1}) - \alpha \ln \pi_\phi(a_{t+1} | s_{t+1}) \right) \right)^2 \right] \quad (8.11)$$

其中 $\mathcal{D}$ 是一个 replay buffer。这里同样使用了目标网络 (θ) 和主网络 (ψ) 两套神经网络, 但是网络的参数更新的方式不同, SAC 中使用了 exponentially moving average 方法更新目标网络的参数, 其效果是可以对近期的数据赋予了更大的权重, 而对较早的数据赋予较小的权重, 平滑目标网络的更新, 使得训练更加稳定。在每个梯度反向传播参数更新的步骤中增加了额外的一步参数更新 $\theta \leftarrow \tau\psi + (1 - \tau)\theta$ 不过论文中指出 Q-learning 中使用的周期性的参数更新方式也是一种选择, 并给出了实验对比结果。对于策略更新, 目标是最小化当前策略分布 π' 与目标分布之间的 KL 散度:

$$\pi_{\text{new}} = \arg \min_{\pi'} D_{KL} \left(\pi'(\cdot | s) \parallel \underbrace{\frac{\exp(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(s, \cdot))}{Z^{\pi_{\text{old}}}(s, \cdot)}}_{\text{target distribution}} \right)$$

即最小化 KL 散度的期望

$$\begin{aligned} J_\pi(\phi) &= \mathbb{E}_{s_t \sim \mathcal{D}} \left[D_{KL} \left(\pi_\phi(\cdot | s_t) \parallel \frac{\exp Q_\psi(s_t, \cdot)}{Z_\psi(s_t)} \right) \right] \\ &= \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\phi} \left[\ln \left(\frac{\pi_\phi(a_t | s_t)}{\exp(\frac{1}{\alpha} Q_\psi(s_t, a_t) - \ln Z(s_t))} \right) \right] \\ &= \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\phi} \left[\ln \pi_\phi(a_t | s_t) - \frac{1}{\alpha} Q_\psi(s_t, a_t) + \ln Z(s_t) \right] \end{aligned} \quad (8.12)$$

算法 20: Soft Actor-Critic

初始化: 初始化两套价值网络参数 ψ_1, ψ_2 ; 初始化策略网络参数 ϕ ; 初始化两套目标价值网络参数 $\theta_1 \leftarrow \psi_1, \theta_2 \leftarrow \psi_2$; 初始化空的回放区 $\mathcal{D}$ 。

目标 : 学习到最优策略以最大化等式(8.11)定义的 $J_Q(\psi)$ 。

```

1 for 每一次迭代 do
2   for 每个特定环境中的时间步 do
3      $a_t \sim \pi_\phi(a_t | s_t);$ 
4      $s_{t+1} \sim p(s_{t+1} | s_t, a_t);$ 
5      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1})\};$ 
6   end
7   for 每次梯度反向传播 do
8      $\psi_i \leftarrow \psi_i - \lambda_Q \hat{\nabla}_{\psi_i} J_Q(\psi_i), \text{ for } i \in \{1, 2\};$  // 更新主网络参数。
9      $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_{\phi_i} J_\pi(\phi);$  // 更新策略网络参数。
10     $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha J(\alpha);$  // 调整温度参数。
11     $\theta_i \leftarrow \tau \psi_i + (1 - \tau) \theta_i, \text{ for } i \in \{1, 2\};$  // 更新目标网络参数。
12  end
13 end
```

以往我们通过概率 $\pi(a | \pi)$ 采样动作，这样的做法会使得梯度的计算变得不可微，SAC 使用了重参数化技巧 (reparameterization trick) 来生成动作

$$a_t = f_\phi(\epsilon_t; s_t) = f_\phi^\mu(s_t) + \epsilon_t \odot f_\phi^\sigma(s_t)$$

其中, $f_\phi^\mu(s_t)$ 是策略网络的输出的均值, 表示在状态 s_t 下的动作分布的均值; $f_\phi^\sigma(s_t)$ 是策略网络的输出的方差, 表示在状态 s_t 下的动作分布的方差; ϵ_t 是从标准正态分布 $\mathcal{N}(0, 1)$ 中采样的噪声; $\odot$ 表示逐元素乘法。再去掉不影响梯度值的常量 $\ln Z(s_t)$, 优化目标 J_π 转化成

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}} \left[\ln \pi_\phi(f_\phi(\epsilon_t; s_t) | s_t) - \frac{1}{\alpha} Q_\theta(s_t, f_\phi(\epsilon_t; s_t)) \right] \quad (8.13)$$

其随机梯度为

$$\hat{\nabla}_{\phi} J_{\pi}(\phi) = \nabla_{\phi} \ln \pi_{\phi}(a_t | s_t) + (\nabla_{a_t} \ln \pi_{\phi}(a_t | s_t) - \frac{1}{\alpha} \nabla_{a_t} Q(s_t, a_t)) \nabla_{\phi} f(\epsilon_t; s_t) \quad (8.14)$$

其中 a_t 由上述重参数化技巧估算。

[6] 的 SAC 算法伪代码见算法20，不过当前工作的核心是算法的数学原理，所以并没有使用温度参数的自动调整；同时为了训练的稳定，该论文还使用了**双值 Q 网络**的方法，这里不展开讨论。

在实现（参考）中还需要注意的地方是对策略网络采样动作的时候，如果需要映射到动作空间的取值范围所使用的映射函数是非线性的，例如 $\tanh(\cdot)$ ，那么会造成 a_t 的实际概率密度会发生变化，此时需要再用到概率密度变换定理进行修正。

9 RLHF

在正式讨论 RLHF 之前，需要简单概括大语言模型（LLM）进行训练所需要经历的三个环节，以便于理解为什么要使用 RLHF 以及 RLHF 是基于什么样的过程发挥了怎样的作用。随后会结合 Actor-Critic 的强化学习算法对 RLHF 的原理进行介绍，解读其状态空间、动作空间、即时奖励、策略函数、状态价值函数、目标函数及其优化的随机过程。有了这些理论基础以后，本章会从工程实践的角度解读在开源社区比较受欢迎的 RLHF 框架：OpenRLHF[7]。

9.1 LLM 的训练环节

LLM 的训练环节主要分为预训练、有监督微调、人类反馈强化学习这三个阶段 [8]，如图2所示，RLHF 本身又分为训练奖励模型、优化 LLM 的输出两个阶段。

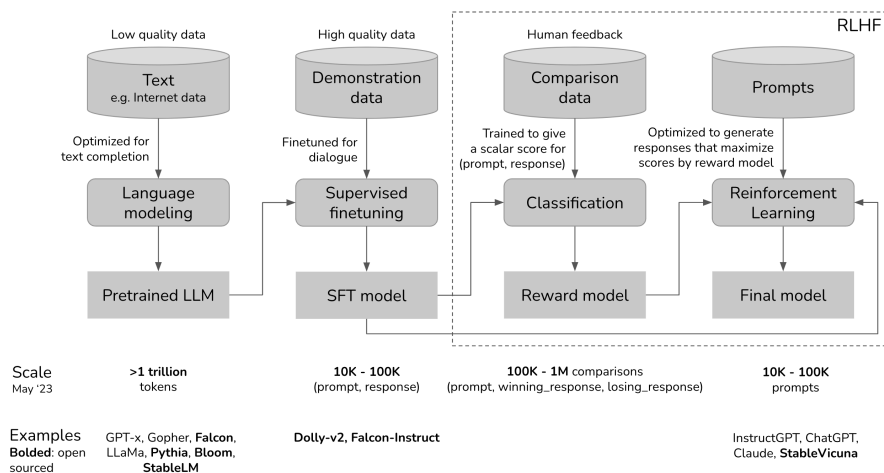


图 2: LLM 的三个训练阶段 [8]

9.2 RLHF 的数学原理

在之前的章节中我们介绍了结合策略更新与价值更新的 Actor-Critic 算法，其中 A2C 使用了时间差分的思想，这里进一步介绍 A3C——Asynchronous Advantage Actor-Critic 算法 [9]。该算法在 A2C 的基础上使用多线程的方式模拟随机过程，然后借助于 Hogwild![10] 证明的异步 SGD 的有效性，加速了学习过程。

9.2.1 状态空间与动作空间的定义

- 状态（空间）：文本前缀 $\{\text{prompt}, a_{1:t-1}\}$ 或者其对应的模型隐层变量 h_{t-1} 。
- 动作（空间）：从“词汇表”中选择的下一个 token a_t 。
- 状态转移：自回归生成新的 token 构成新的状态。

9.2.2 Reward Model

即时奖励 r_{i+1} 来自于 reward model, 这个奖励模型通过最大化人类的正偏好和负偏好的打分来训练的。原文中给出了两种奖励模型的训练方式——Clipped Surrogate Objective 和 Adaptive KL Penalty Coefficient, 并指出在他们的实验中, 前者的效果更好 [11]。但是最近几年业界（主要指大模型领域）主要使用后者, 其收敛性、对旧策略的惩罚的衡量、与人类偏好的对齐的能力更佳, clipped 方法在小 RL 环境中效果比较好。这里主要介绍带 KL 惩罚项的优化目标:

$$J(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}(\pi_{\theta_{\text{old}}}(\cdot | s_t) \parallel \pi_\theta(\cdot | s_t)) \right] \quad (9.1)$$

其中, π_θ 是新的策略模型, $\pi_{\theta_{\text{old}}}$ 是参数更新前的策略模型, $\hat{A}_t$ 是 Advantage (时间差分) 的估计。在具体的工程实践中, 考虑到计算成本, 通常不会严格按照数学定义计算 KL 散度, 而是仅在当前生成的 token 上进行的估算, 细节如下:

- 理论上

$$\text{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta) = \sum_i^V \pi_\theta^t(a_i | s_t) (\ln \pi_\theta^t(a_i | s_t) - \ln \pi_{\theta_{\text{old}}}^t(a_i | s_t))$$

其中 V 是语料库大小。

- 工程上

$$\text{KL}_t(\pi_{\theta_{\text{old}}} \parallel \pi_\theta) = \ln \pi_\theta^t(a_t | s_t) - \ln \pi_{\theta_{\text{old}}}^t(a_t | s_t)$$

仅使用当前生成的 token 计算新老策略函数预测概率的差值。

9.2.3 Actor

在有了可以给出即时奖励的 Reward Model 之后，我们便可以使用时间差分 (Advantage)

$$\delta_t = R_{i+1} - V(s_i; \theta')$$

对策略模型进行参数更新，也就是算法21中的第 17 行中描述的对 θ' 进行更新的过程。

9.2.4 Critic

使用时间差分对价值模型进行更新——对应算法21中的第 17 行中描述的对 w' 进行更新的过程。

算法 21: Asynchronous Actor-Critic (A3C)——对每个线程

初始化：全局共享的策略函数 $\pi(a | s, \theta)$ ；全局共享的状态价值函数 $v(s, w)$ ；全局计数器 T ；每个线程自己的待优化参数 θ' 和 w'

目标：学习到一个最优的策略以最小化损失 L

```

1 repeat
2   梯度置零： $\nabla_{\theta} \leftarrow \mathbf{0}$ 、 $\nabla_w \leftarrow \mathbf{0}$ ；
3   对不同线程同步共享参数： $\theta' = \theta$ 、 $w' = w$ ；
4    $t_{start} = t$ ；
   // 当前状态是文本前缀或者对应的模型隐层激活
5   获得当前状态  $s_t$ ；
6   repeat
7     从策略  $\pi(a_t | s_t; \theta')$  生成动作  $a_t$ ；
8     观察到奖励  $r_{t+1}$  和新的状态  $s_{t+1}$ ；
9      $t \leftarrow t + 1$ ；
10     $T \leftarrow T + 1$ ；
11  until 终点状态  $s_t$  或者  $t - t_{start} = t_{max}$ ；
12   $R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta') & \text{for non-terminal } s_t \end{cases}$ ；
13  for  $i \in \{t - 1, \dots, t_{start}\}$  do
14     $R_{i+1} \leftarrow r_{i+1} + \gamma R_i$ ；
15    对  $\theta'$  进行梯度累加： $\nabla_{\theta'} \leftarrow \nabla_{\theta'} + \nabla_{\theta'} \ln \pi(a_i | s_i; \theta') (R_{i+1} - V(s_i; w'))$ ；
16    对  $\nabla_{w'}$  进行梯度累加： $\nabla_{w'} \leftarrow \nabla_{w'} + \partial (R_{i+1} - V(s_i; w')) / \partial w'$ ；
17    基于 Hogwild! 证明的异步 SGD 有效性结论，使用  $\nabla_{\theta'}$  对  $\theta'$  进行更新、
      使用  $\nabla_{w'}$  对  $w'$  进行更新。；
18  end
19 until  $T > T_{max}$ ；
20 return  $v_k, \pi_k$ ；

```

9.3 PPO

这里暂时只接给出 PPO[12] 算法的优化目标，后面会细节：

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{\text{old}}}(O | q)]$$

$$\frac{1}{|O|} \sum_{t=1}^{|O|} \left[\frac{\pi(o_t | q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t | q, o_{<t})} A_t, \text{clip} \left(\frac{\pi(o_t | q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t | q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right] \quad (9.2)$$

其中 A_t 通过 GAE (Generalized Advantage Estimation) 方法计算：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t^{\text{GAE}} = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

$$\hat{A}_t^{\text{GAE}} = \delta_t + (\lambda\gamma)\hat{A}_{t+1}^{\text{GAE}}$$

编程计算的时候从后往前算。

9.4 GRPO

GRPO (Iterative Group Relative Policy Optimization) 算法 [12] 伪代码为

算法 22: 迭代式组相对策略优化**Input:** 初始策略模型 $\pi_{\theta_{\text{init}}}$; 奖励模型 r_{φ} ; 任务提示 $\mathcal{D}$; 超参数 ε, β, μ **Output:** 策略模型 π_{θ} **Initialization:** 策略模型 $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$

```

1 for 迭代  $I = 1$  to  $I$  do
2   参考模型  $\pi_{\text{ref}} \leftarrow \pi_{\theta}$ ;
3   for 步骤  $M = 1$  to  $M$  do
4     采样一批数据  $\mathcal{D}_b$  从  $\mathcal{D}$ ;
5     更新旧策略模型  $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ ;
6     对  $\mathcal{D}_b$  中的每个问题  $q$ , 采样  $G$  个输出  $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)$ ;
7     通过运行  $r_{\varphi}$  计算每个采样输出  $o_i$  的奖励  $\{r_i\}_{i=1}^G$ ;
8     通过组相对优势估计, 计算  $o_i$  的第  $t$  个 token 的优势  $\hat{A}_t$ ;
9     for GRPO 迭代  $j = 1$  to  $\mu$  do
10      通过最大化 GRPO 目标 (等式(9.3)) 来更新策略模型  $\pi_{\theta}$ ;
11    end
12    通过使用重放机制的持续训练来更新  $r_{\varphi}$ ;
13  end
14 end

```

$$\begin{aligned}
\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}} (O | q) \right] \\
\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})} \hat{A}_i, \right. \right. \\
\left. \left. \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_i \right] - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} || \pi_{\text{ref}}] \right\} \quad (9.3)
\end{aligned}$$

说明一下上面等式中的 $\hat{A}_i$ 在原论文中是 $\hat{A}_{i,t}$, 我将这里的 t 下标去掉是因为 $\hat{A}_i$ 是整个序列共享的, 不是逐个词元计算的, 原论文的写法不合理。

上述等式中的 KL 散度惩罚模型参数更新的力度, 避免模型参数在强化学习后训练的时候更新太多, 论文 [12] 使用的是 KL 散度的无偏估计:

$$\mathbb{D}_{\text{KL}}[\pi_{\theta} || \pi_{\text{ref}}] = \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})} - 1$$

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

错误理解：算法22中的 1、3、9 行有三个 for 循环，这不是一定必要的，通常奖励模型稳定后就不会再总是更新，所以在一次训练中 π_{ref} 是不变的。然后这里把轨迹采样和优化放在一起了，实际上我们常常可以将轨迹采样先做完，计算每个组中每条轨迹的奖励，然后计算每条轨迹的奖励、组内优势保存起来。然后优化阶段，在保存好的数据中拿出采样评估阶段计算的结果来训练模型。所以实际计算的时候，等式(9.3)中的 π_{θ} 和 $\pi_{\theta_{\text{old}}}$ 不是同时计算的。现在假设我有了一个 SFT 好的模型 π_{θ} 以及处理好的训练数据，此时再实例化一个 π_{ref} 、 $\pi_{\theta_{\text{old}}}$ ，然后我对训练数据进行生成采样和评估，按照 $\langle G, S \rangle$ 这样的维度保存数据，此时 $\pi_{\theta_{\text{old}}} = \pi_{\theta}$ 、 $\pi_{\text{ref}} = \pi_{\theta}$ 。然后进入模型优化阶段，从保存好的数据中选择 batch 个样本，计算 π_{θ} 、 π_{ref} ，然后和已经计算好的 $\pi_{\theta_{\text{old}}}$ 一起计算 $\mathcal{J}_{\text{GRPO}}(\theta)$ ，更新 π_{θ} ，每个 batch 都更新，一共更新若干个 epoch。此时，就看要不要对当前的训练数据进行第二次采样评估，或者更新数据了后再进行采样评估（或者如果此时的 π_{θ} 就直接拿去用了）。然后我们可以进入第二阶段，这个时候需要更新一下 $\pi_{\theta_{\text{old}}}$ ，然后重复上述过程。

GRPO 既可以使用 on-policy 版本也可以使用 off-policy 版本，这里分别介绍两个版本的训练过程：

on-policy 版本每次从处理好的训练集中取出一个批次的数据，然后对这个批次的数据采样生成轨迹，每组采样 G 个，接着计算 GRPO 损失，更新 π_{θ} 。注意使用每个批次的数据进行训练的时候，都会进行 $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ ，所以 Plan-R1[13] 中的 $\pi_{\theta}/\pi_{\theta_{\text{old}}}$ 始终为 1，也就是算法22的第 9 行 $\mu = 1$ ——如果 $\mu > 1$ ，又有点像 $\pi_{\text{behavior}} = \pi_{\theta_0}$ 的 off-policy。

off-policy 版本需要冻结一份行为策略 π_{behavior} ，训练的时候需要使用 π_{behavior} 计算 $\pi_{\theta_{\text{old}}}$ ，或者提前生成好保存起来， π_{behavior} 是始终不变的。但是需要主要 off-policy 是非常不稳定的，训练很容易崩，通常还是使用 on-policy 的版本。

9.5 DPO

DPO 损失的表达式为

$$\begin{aligned}\mathcal{L} &= -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right] \\ &= -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta (\log \pi_\theta^+ - \log \pi_\theta^- - (\log \pi_{\text{ref}}^+ - \log \pi_{\text{ref}}^-)) \right) \right]\end{aligned}\quad (9.4)$$

通常可以将 $\beta (\log \pi_\theta - \log \pi_{\text{ref}})$ 看作偏好或偏恶奖励。DPO 效果如果好的话，偏好奖励会增加，偏恶奖励会减小，有时会观察到都减小，这个没有本质影响，只要偏好奖励下降得比偏恶奖励小就行。DPO 训练通常需要观测 4 个指标：chosen reward、rejected reward、margin = chosen reward - rejected reward、accuracies = $\sum \mathbb{I}(\log y_w > \log y_l) / N$ 。

9.6 RLHF 的工程实践——OpenRLHF

OpenRLHF 解决的关键问题是将上述 actor、critic、reward、reference（通过有监督微调获得的模型）部署在不同的 GPU 上，以支持大参数规模的模型训练。

TODO，总结 OpenRLHF 使用的参数训练技巧：

1. 使用 PPO 中的预训练损失项。
2. 奖励归一化。
3. 基于全局统计的分布式优势归一化。
4. 线性预热余弦退火学习率调度。
5. 使用奖励模型的权重初始化。
6. 对 Actor 使用较低学习率，对 Critic 使用较高学习率。
7. 在训练初期冻结 Actor 权重。
8. 使用 GAE（Generalized Advantage Estimation）。

10 强化学习实践指南

TODO

算法 23: 在线 AI 反馈 (OAIF) 直接偏好对齐 (DAP) 算法

Input: 训练步数 T

提示数据集 $\mathbb{D}_{\mathcal{X}} = \{x_i\}_{i=1}^N$

监督微调基线模型 π_{θ^0}

LLM 标注器

DAP 损失函数 $\ell(x, y^+, y^-, \theta)$

Output: 对齐后的语言模型（策略） π_{θ^T}

```

1 for  $t \leftarrow 0$  to  $T$  do
2   采样提示  $x \sim \mathbb{D}_{\mathcal{X}}$ ;
3   采样响应对  $y^1, y^2 \sim \pi_{\theta^t}(\cdot|x)$ ;
4   使用 LLM 标注器获取偏好对  $y^+, y^-$ ;
5   使用  $\nabla_{\theta} \ell(x, y^+, y^-, \theta^t)$  更新  $\theta^t$  到  $\theta^{t+1}$ ;
6 end
```

参考文献

- [1] Shiyu Zhao. *Mathematical Foundations of Reinforcement Learning*. Springer Verlag, Singapore, 2025.
- [2] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [3] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- [4] OpenAI. Spinning up in deep reinforcement learning. <https://spinningup.openai.com/en/latest/algorithms/sac.html>, 2018. Accessed: 2025-05-12.
- [5] Tuomas Haarnoja Haoran Tang. Learning diverse skills via maximum entropy deep. <https://bair.berkeley.edu/blog/2017/10/06/soft-q-learning/>, 2017. Accessed: 2025-05-15.
- [6] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.
- [7] Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.
- [8] Chip Huyen. Rlhf: Reinforcement learning from human feedback. https://huyenchip.com/2023/05/02/rlhf.html#rlhf_overview, 2023. Accessed: 2025-05-13.

- [9] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR.
- [10] Benjamin Recht, Christopher Re, Stephen Wright, and Feng Niu. Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in neural information processing systems*, 24, 2011.
- [11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [13] Xiaolong Tang, Meina Kan, Shiguang Shan, and Xilin Chen. Plan-r1: Safe and feasible trajectory planning as language modeling. *arXiv preprint arXiv:2505.17659*, 2025.
- [14] Koundinya Vajjha, Barry Trager, Avraham Shinnar, and Vasily Pestun. Formalization of a stochastic approximation theorem. *arXiv preprint arXiv:2202.05959*, 2022.
- [15] 赛亚茂. 强化学习中的重要收敛性结论. <https://blog.csdn.net/shengzimao/article/details/130939571>, 2023. Accessed: 2025-05-19.

A 附录

附录中总结了前面章节中需要用到的定理及其证明, 严格来说, 对强化学习算法的创新和改动都需要重新审查这些定理条件是否满足。该部分内容主要参考 [1, 14, 15]。

A.1 收缩映射定理

定理 A.1 (收缩映射定理). 对于任意的形如 $x = f(x)$ 的等式, 其中 x 和 $f(x)$ 都是实向量, 如果 $f(\cdot)$ 是一个收缩映射, 那么将有如下性质:

1. 存在性: 存在一个固定点 x^* 满足 $f(x^*) = x^*$ 。
2. 唯一性: 固定点 x^* 是唯一的。
3. 算法: 考虑迭代过程

$$x_{k+1} = f(x_k)$$

其中 $k = 0, 1, 2, \dots$ 。当 $k \rightarrow \infty$ 的时候, $x_k \rightarrow x^*$, 并且收敛的速度是指数级的。

贝尔曼最优方程的简洁矩阵形式为

$$v = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v)$$

其中 $v \in \mathbb{R}^{|S|}$ 且 $\max_\pi$ 是在每个元素上操作的, r_π 和 P_π 的结构和常规的贝尔曼方程相同:

$$[r_\pi]_s = \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{r \in \mathcal{R}} p(r | s, a) r, \quad [P_\pi]_{s, s'} = p(s' | s) \doteq \sum_{a \in \mathcal{A}} \pi(a | s) p(s' | s, a)$$

再定义 $f(v) \doteq \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v)$, 于是贝尔曼最优方程可以表达为

$$v = f(v)$$

贝尔曼最优方程的收缩映射性质 上述贝尔曼最优方程右边的函数 $f(v)$ 是一个收

缩映射。特别的，对于任意的

$$v_1, v_2 \in \mathbb{R}^{|S|}$$

有以下不等式成立

$$\|f(v_1) - f(v_2)\|_\infty \leq \gamma \|v_1 - v_2\|_\infty$$

其中 $\gamma \in (0, 1)$ 为折扣率， $\|\cdot\|_\infty$ 是最大范数，表示一个向量所有元素的绝对值的最大值。同时满足上述收缩映射定理的存在性与唯一性以及求解算法。

证明. TODO. ■

A.2 Dvoretzky's 收敛定理

定理 A.2 (Dvoretzky's 收敛定理). 考虑一个随机过程 $\Delta_{k+1} = (1 - \alpha_k)\Delta_k + \beta_k\eta_k$ ，其中 $\{\alpha_k\}_{k=1}^\infty$ ， $\{\beta_k\}_{k=1}^\infty$ ， $\{\eta_k\}_{k=1}^\infty$ 是随机序列。当满足以下条件：

1. $\sum_{k=1}^\infty \alpha_k = \infty$, $\sum_{k=1}^\infty \alpha_k^2 < \infty$, $\sum_{k=1}^\infty \beta_k^2 < \infty$ *uniformly almost surely*;
2. $\mathbb{E}[\eta_k | H_k] = 0$ and $\mathbb{E}[\eta_k^2 | H_k] \leq C$ *almost surely*;

证明. 假设 α_k, β_k 可以由 H_k 完全确定，即 $\alpha_k = \alpha_k(H_k)$ ， $\beta_k = \beta_k(H_k)$ 。则：

$$\mathbb{E}[\alpha_k | H_k] = \alpha_k, \quad \mathbb{E}[\beta_k | H_k] = \beta_k$$

构造 $h_k = \Delta_k^2$ ，可得

$$\begin{aligned} \mathbb{E}[h_{k+1} - h_k | H_k] &= \mathbb{E}[\Delta_{k+1}^2 - \Delta_k^2 | H_k] \\ &= \mathbb{E}[-\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2\eta_k^2 + (2 - 2\alpha_k)\beta_k\eta_k\Delta_k | H_k] \\ &= -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2\mathbb{E}[\eta_k^2 | H_k] + (2 - 2\alpha_k)\beta_k \end{aligned}$$

因为 $\sum_{k=1}^\infty \alpha_k^2 < \infty$ ，所以 $\alpha_k \rightarrow 0$ 。

所以存在 N ，当 $k > N$ 时有 $\alpha_k \leq |\alpha_k| < 1$ (极限定义)， $-\alpha_k(2 - \alpha_k)\Delta_k^2 < 0$ ，

此时 $\mathbb{E}[h_{k+1} - h_k | H_k] \leq \beta_k^2 C, k > N$ 。又因为 $\sum_{k=1}^{\infty} \beta_k^2 = C_{\beta^2} < \infty$ ，因此有：

$$\begin{aligned} \sum_{k=1}^{\infty} \mathbb{E}[h_{k+1} - h_k | H_k] &= \left(\sum_{k=1}^N + \sum_{k=N+1}^{\infty} \right) \mathbb{E}[h_{k+1} - h_k | H_k] \\ &\leq \sum_{k=1}^N \mathbb{E}[h_{k+1} - h_k | H_k] + \sum_{k=N+1}^{\infty} \beta_k^2 C \\ &\leq \sum_{k=1}^N \mathbb{E}[h_{k+1} - h_k | H_k] + C_{\beta^2} C < \infty \end{aligned}$$

又因为

$$\begin{aligned} \sum_{k=1}^{\infty} \alpha_k \Delta_k^2 &= \sum_{k=N}^{\infty} \alpha_k \Delta_k^2 + \sum_{k=1}^N \alpha_k \Delta_k^2 < \sum_{k=1}^N \alpha_k \Delta_k^2 + \sum_{k=N}^{\infty} \alpha_k (2 - \alpha_k) \Delta_k^2 \\ &< \sum_{k=1}^N \alpha_k \Delta_k^2 + \sum_{k=1}^{\infty} \alpha_k (2 - \alpha_k) \Delta_k^2 \\ &< \sum_{k=1}^N \alpha_k \Delta_k^2 - \sum_{k=1}^{\infty} \mathbb{E}[h_{k+1} - h_k | H_k] + \sum_{k=1}^{\infty} \beta_k^2 C \end{aligned}$$

由之前的证明的 $\mathbb{E}[h_{k+1} - h_k | H_k] < \infty$ ， $\sum_{k=1}^{\infty} \beta_k^2 C = C_{\beta^2} C < \infty$ 可知：

$$\sum_{k=1}^{\infty} \alpha_k \Delta_k^2 < \infty$$

因此几乎必然有 $\Delta_k \rightarrow 0$ ，证明完毕。 ■

A.3 Robbins-Monro 定理

定理 A.3 (Robbins-Monro 定理). 对于求解 $g(w) = 0$ 的迭代算法

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k) \quad k = 1, 2, 3, \dots$$

其中 w_k 是对方程根的第 k 个估计， $\tilde{g}(w_k, \eta_k)$ 是第 k 个带噪声的观测， α_k 是一个正系数。如果下列条件

1. $0 < c_1 \leq \nabla_w g(w) \leq c_2$ for all w

$$2. \sum_{k=1}^{\infty} a_k = \infty \quad \text{and} \quad \sum_{k=1}^{\infty} a_k^2 < \infty$$

$$3. \mathbb{E}[\eta_k | H_k] = 0 \quad \text{and} \quad \mathbb{E}[\eta_k^2 | H_k] < \infty$$

满足, 其中 $\mathcal{H} = \{w_k, w_{k-1}, \dots\}$, 那么 w_k 几乎必然收敛到根 w^* 满足 $g(w^*) = 0$ 。该算法称为 *Robbins-Monro* 算法。

证明. 上述 Robbins-Monro 算法可以进一步写成

$$\begin{aligned} w_{k+1} &= w_k - a_k \tilde{g}(w_k, \eta_k) \\ &= w_k - a_k [g(w_k) - \eta_k] \end{aligned}$$

从而

$$w_{k+1} - w^* = w_k - w^* - a_k [g(w_k) - g(w^*) - \eta_k]$$

又根据均值定理, 我们有

$$g(w_k) - g(w^*) = \nabla_w g(w'_k)(w_k - w^*)$$

其中 $w'_k \in [w_k, w^*]$ 。令 $\Delta_k \doteq w_k - w^*$ 。上面的等式变成

$$\begin{aligned} \Delta_{k+1} &= \Delta_k - a_k [\nabla_w g(w'_k)(\Delta_k) + \eta_k] \\ &= \Delta_k - a_k \nabla_w g(w'_k) \Delta_k + a_k (-\eta_k) \\ &= [1 - \underbrace{a_k \nabla_w g(w'_k)}_{\alpha_k}] \Delta_k + a_k (-\eta_k) \end{aligned}$$

注意根据定理假设条件 $\nabla_w g(w)$ 是有上下界的, 满足 $0 < c_1 \leq \nabla_w g(w) \leq c_2$, 又因为有定理假设条件 2, 那么 Dvoretzky's 定理的所有条件都满足, 因此 $\lim_{k \rightarrow \infty} \Delta_k = 0$ a.s.。 ■

A.3.1 应用

Robbins-Monro 定理可以用来估计数学期望, 这在强化学习中经常用到。可以构造 $g(w) \doteq w - \mathbb{E}[X]$, 此时问题变成求解方程 $g(w) = 0$ 。给定一个值 w , 带噪声

的观察为 $\tilde{g} \doteq w - x$ ，其中 x 是对 X 的采样。 $\tilde{g}$ 可以写成

$$\begin{aligned}\tilde{g}(w, \eta) &= w - x \\ &= w - x + \mathbb{E}[X] - \mathbb{E}[X] \\ &= (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta\end{aligned}$$

其中 $\eta \doteq \mathbb{E}[X] - x$ 。此时估算期望的 Robbins Monro 算法为

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k)$$

A.4 拓展 Dvoretzky's 收敛定理

拓展的 Dvoretzky's 定理是一个可以处理多变量的更通用的定理，可以被用来分析随机迭代算法的收敛性，在 Q-learning 中会有应用。

定理 A.4 (拓展 Dvoretzky's 收敛定理). 考虑一个实数有限集合 $\mathcal{S}$ ，对于随机过程

$$\Delta_{k+1}(s) = (1 - \alpha_k(s))\Delta_k(s) + \beta_k(s)\eta_k(s)$$

对于每一个 $s \in \mathcal{S}$ ， $\Delta_k(s)$ 将几乎必然收敛到 0 只要以下条件满足：

1. $\sum_{k=1}^{\infty} \alpha_k(s) = \infty$, $\sum_{k=1}^{\infty} \alpha_k^2(s) < \infty$, $\sum_{k=1}^{\infty} \beta_k^2(s) < \infty$
and $\mathbb{E}[\beta_k^2(s) | \mathcal{H}_k] \leq \mathbb{E}[\alpha_k^2(s) | \mathcal{H}_k]$ uniformly almost surely
2. $\|\mathbb{E}[\eta_k^2 | \mathcal{H}_k]\|_{\infty} \leq \gamma \|\Delta_k\|_{\infty}$, 其中 $\gamma \in (0, 1)$
3. $\text{var}[\eta_k(s) | \mathcal{H}_k] \leq C(1 + \|\Delta_k(s)\|_{\infty})^2$, 其中 C 为一个常数

这里 $\mathcal{H}_k = \{\Delta_k, \Delta_{k-1}, \dots, \eta_{k-1}, \dots, \alpha_{k-1}, \dots, \beta_{k-1}, \dots\}$ 代表历史信息， $\|\cdot\|_{\infty}$ 是最大范数，表示一个向量所有元素的绝对值的最大值。

有以下要点需要注意

- 拓展的 Dvoretzky's 定理可以处理多个状态的强化学习问题。
- 在应用定理的时候，我们需要明确指出定理条件在集合 $\mathcal{S}$ 中的每一个状态（或者状态动作对）上都成立。

证明. TODO. ■

A.5 随机梯度下降 (SGD)

随机梯度下降法的核心思想是用随机的梯度代替真实的梯度。

定理 A.5. 对于最优化问题 $\min_w J(w) = \mathbb{E}[f(w, X)]$, 当使用随机过程迭代式 $w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k)$ 进行参数迭代时, 若满足

$$\begin{aligned} (1) \quad & 0 < c_1 \leq \nabla_w^2 f(w, X) \leq c_2; \\ (2) \quad & \sum_{k=1}^{\infty} \alpha_k = \infty, \quad \sum_{k=1}^{\infty} \alpha_k^2 < \infty; \\ (3) \quad & \{x_k\}_{k=1}^{\infty} \text{ i.i.d} \end{aligned}$$

则 $w_k \rightarrow w^*$, 其中 $\nabla_w \mathbb{E}[f(w^*, X)] = 0$ a.s.

证明. TODO. ■

A.6 马尔可夫过程的稳态分布定理

稳态分布定理的核心含义是, 在满足条件的马尔可夫链中, 随着时间的推移, 链的状态分布会逐渐趋向于一个固定的分布, 且这个分布与初始状态无关。有以下要点:

1. **时间无关的长期行为** 无论马尔可夫链的初始状态是什么, 经过足够长的时间, 系统将不再依赖于起始状态, 且进入长期的平稳状态。换句话说, 系统最终会在各个状态之间按照稳态分布频率进行转移。
2. **不可约性和遍历性** 定理要求马尔可夫链是不可约的, 即从任意一个状态可以通过有限步到达任意其他状态 (状态之间可以相互连通)。还要求链是正则或遍历的, 即所有状态都可以在有限时间内返回自身。这两个性质保证了系统不会停留在某个特定状态或某个状态集合中。
3. **稳态分布的物理解释** 在很多实际应用中, 稳态分布代表了系统在长期运行中的各状态的“停留频率”或“访问概率”。例如:
 - 在排队论中, 稳态分布可以描述在长期运行的服务系统中, 系统处于不同状态的概率。

- 在强化学习中，稳态分布可以表示在马尔可夫决策过程 (MDP) 中，某一策略下的状态分布，这对于理解策略的长期效果很重要。

4. **计算稳态分布** 稳态分布 π 可以通过求解方程组 $\pi P = \pi$ 和 $\sum_i \pi_i = 1$ 来计算。对于大规模的状态空间，常用数值方法来近似求解，比如迭代法或幂法 (Power Iteration)。

定理 A.6 (马尔可夫过程的稳态分布定理). 设 *Markov Process* 的状态空间为 $\mathcal{S}$, 状态量的个数为 S , 定义在策略 π 下的状态转移概率矩阵 $P_\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$, 定义 k 步状态转移矩阵 $P_\pi^k = \left\{ p_{i,j}^{(k)} \right\}_{|\mathcal{S}| \times |\mathcal{S}|}$

$$p_{ij}^{(k)}(\pi) = \mathbf{Prob}(S_{t_k} = j \mid S_{t_0} = i, \pi)$$

其满足 $P_\pi^k = P_\pi P_\pi^{k-1}$ 。对于任意一个初始状态分布 $d_0 \in \mathbb{R}^{|\mathcal{S}|}$, 在策略 π 下经过 k 轮迭代后的状态分布为 d_k , 且有 $d_k^\top = d_0^\top P_\pi^k$ 。若对于任意的两个状态 $s_i, s_j \in \mathcal{S}$, 都存在有限的步长 k , 使得 $[P_\pi^k]_{ij} > 0$, 则有如下结论:

$$\begin{aligned} (1) \quad & P_\pi^k \rightarrow \frac{1}{|\mathcal{S}|} d_\pi^\top; \\ (2) \quad & d_\pi^k \rightarrow d_\pi^0 \frac{1}{|\mathcal{S}|} d_\pi^\top = d_\pi^\top; \\ (3) \quad & d_\pi^\top \text{ 满足 } d_\pi^\top = d_\pi^\top P. \end{aligned}$$

此时称这样的马尔可夫过程是正则的。